

Data Governance Copilot

Interview Question Bank

Topic 13 — AWS Deployment & Infrastructure

12 questions · 4 sections · ECS Fargate · ECR · RDS pgvector · ElastiCache · ALB/WAF · Bedrock · Secrets Manager · X-Ray

Foundational Core concepts **Intermediate** Design decisions **Advanced** Deep internals **Gotcha** Real bugs / traps

ECS Fargate, ECR & Container Architecture

Q01**Foundational**

Explain your ECS Fargate deployment architecture. How many task definitions do you have, what runs in each, and how does traffic reach them?

Hint: Walk the full request path from internet to ECS task.

Key points to cover:

- Three ECS task definitions: app (Streamlit UI, port 8501), api (FastAPI/uvicorn, port 8000), mcp_server (MCP SSE, port 8002)
- Each task runs a single container built from the same Dockerfile — different ENTRYPOINT/CMD per task definition
- Fargate: AWS manages the underlying EC2 host — you specify vCPU and memory (e.g. 1 vCPU / 2GB for api, 2 vCPU / 4GB for app with LangGraph in-process)
- Traffic path: Internet → Route 53 → ALB → Target Group → ECS task private IP (ENI in VPC subnet)
- ALB listener rules: path-based routing — /query*, /health, /agents/status → api target group; /* → app target group; :8002 → mcp_server target group
- ECS Service: manages desired task count (e.g. 2 replicas for api), rolling update deployment, health check integration with ALB
- Service discovery: ECS tasks register with ALB target group automatically via aws_alb_target_group_attachment — no manual IP management
- VPC: tasks run in private subnets — no public IP; ALB is in public subnet; NAT gateway for outbound calls (Groq API, Collibra, Jira)

Q02**Intermediate**

Walk through your multi-stage Dockerfile. What does each stage do and why is the two-stage build important for production image size?

Hint: Multi-stage builds are a Docker best practice — explain the specific benefit for a Python ML application.

Key points to cover:

- Stage 1 — builder: FROM python:3.12-slim AS builder; installs uv; runs uv pip install -r requirements.txt into /install
- Builder stage has full build tools (gcc, build-essential) needed to compile psycpg2-binary, numpy, and other C-extension packages
- Stage 2 — runtime: FROM python:3.12-slim (fresh base); COPY --from=builder /install /usr/local/lib/python3.12/site-packages
- Runtime stage has no build tools — only the compiled packages are copied; gcc/build-essential are NOT in the final image
- Size difference: with build tools ~1.2GB; without (multi-stage) ~400MB — significant ECR storage cost and pull time savings
- Security: build tools (gcc, make) in production containers are an attack surface — multi-stage removes them
- PYTHONPATH=./src set in Dockerfile ENV — ensures all src/ imports work without src. prefix inside the container
- ENTRYPOINT varies by task: api uses uvicorn src.api.app:app; app uses streamlit run src/ui/app.py; mcp_server uses python -m src.mcp_server.server

```
# Dockerfile multi-stage pattern FROM python:3.12-slim AS builder WORKDIR /build COPY requirements.txt . RUN pip install uv --break-system-packages && \ uv pip install -r requirements.txt --target /install FROM python:3.12-slim AS runtime WORKDIR /app COPY --from=builder /install /usr/local/lib/python3.12/site-packages COPY src/ ./src/ ENV PYTHONPATH=./src # ENTRYPOINT set per ECS task definition override
```

Q03**Intermediate**

How does ECR fit into your CI/CD pipeline? Walk through the GitHub Actions workflow from code push to running container.

Hint: Trace the full build → push → deploy lifecycle.

Key points to cover:

- GitHub Actions ci.yml triggers on push to main — three jobs: lint (ruff), test (pytest --cov), docker-build-push
- docker-build-push job: configure-aws-credentials action (OIDC role, no long-lived keys); docker/login-action for ECR registry
- Build: `docker build -t {ECR_REGISTRY}/{repo}:{git_sha} .` — tagged with the Git commit SHA for immutable image versioning
- Push: `docker push` to ECR — ECR stores the image in the same AWS region as ECS for fast pull
- Deploy: `aws ecs update-service --force-new-deployment` — ECS pulls the new image and performs rolling update (old tasks drain, new tasks start)
- ECR lifecycle policy: keep last 10 images, expire older than 30 days — prevents unbounded ECR storage cost
- OIDC (OpenID Connect): GitHub Actions assumes an IAM role via OIDC token — no `AWS_ACCESS_KEY_ID` stored in GitHub Secrets; more secure than static keys
- Rollback: if new tasks fail health checks, ECS rolls back to previous task definition automatically; ECR retains the previous image SHA

RDS, ElastiCache & Data Layer

Q04**Intermediate**

Your production database is RDS PostgreSQL 16 with pgvector. Why RDS over Neon in production and how do you size it?

Hint: You use Neon in dev but RDS in prod — explain the architectural reasoning.

Key points to cover:

- Neon in dev: serverless PostgreSQL — scales to zero, cheap, no maintenance; perfect for intermittent dev usage
- RDS in prod: dedicated instance — predictable performance, no cold-start latency, fine-grained control over storage IOPS and memory
- Neon serverless cold start: if the Neon compute is idle and a query arrives, 500ms-2s wake-up latency — unacceptable for production API response times
- pgvector performance: HNSW index scans on large corpora need consistent I/O — RDS provisioned IOPS (io2) guarantees throughput; Neon shared infra does not
- Sizing: `db.r6g.large` (2 vCPU, 16GB RAM, Graviton) — memory-optimised for pgvector index caching; pgvector HNSW `ef_construction` requires significant RAM
- Storage: `gp3` with 100GB allocated, autoscaling to 1TB — pgvector embeddings: $1536 \text{ dims} \times 4 \text{ bytes} \times 100\text{k chunks} = \sim 600\text{MB}$ for embeddings alone
- Multi-AZ: enabled for production — automatic failover to standby replica in < 60s on primary failure
- PO item: separate pgvector RDS instance from transactional/checkpointer RDS — HNSW scans are I/O-heavy and should not compete with checkpoint writes

Q05**Intermediate****Describe your ElastiCache Redis configuration for production. What instance type, replication, and settings matter for your caching use case?**

Hint: Redis in production has specific configuration requirements beyond 'just run Redis'.

Key points to cover:

- Instance: cache.r6g.large (Graviton, memory-optimised) — 13GB RAM; your cached agent results are JSON payloads up to ~100KB each
 - Eviction policy: allkeys-lru — when memory is full, evict least recently used keys; correct for pure cache workload (never noeviction — causes OOM)
 - Persistence: disabled (no RDB, no AOF) — cache doesn't need durability; persistence adds I/O overhead and slows writes
 - Replication: one primary + one read replica — replica provides failover; rate limiter reads can hit replica to reduce primary load
 - Cluster mode: disabled — your workload doesn't require horizontal sharding at this scale; single primary with replica is simpler
 - TLS in transit: enabled on port 6380 — all cache.py and middleware.py connections use ssl=True; REDIS_TLS=true env var
 - AUTH token: enabled — stored in AWS Secrets Manager; injected as REDIS_AUTH env var; prevents unauthenticated connections from within VPC
 - CloudWatch alarms: CacheHitRate (alert if < 60%), Evictions (alert if > 0 sustained), FreeableMemory (alert if < 20%)
-

Q06**Advanced****You have two PostgreSQL needs: checkpointing (transactional) and pgvector (analytical). Why should these be on separate RDS instances and what happens if they share one?**

Hint: This is your P0 production hardening item — explain the I/O contention problem.

Key points to cover:

- HNSW vector search: scans large index structures sequentially — high I/O, high memory bandwidth, long-running queries (10-500ms for large corpora)
 - Checkpoint writes: small, frequent JSONB upserts — require low latency (< 5ms) and high IOPS for responsiveness
 - Shared instance problem: during a busy period, 10 concurrent vector searches saturate the RDS I/O bandwidth — checkpoint writes queue behind them
 - Impact: graph.ainvoke() checkpoint saves take 200ms instead of 5ms — every agent node completion is delayed — total query latency spikes
 - PostgreSQL row locking: HNSW index scans take ShareLock on index pages; concurrent inserts (upserts during ingestion) wait — pgvector and ingestion pipeline contend
 - Fix: separate RDS instances — pgvector on a memory-optimised db.r6g.xlarge (4 vCPU, 32GB RAM); checkpoint on a general-purpose db.t3.medium
 - Connection routing: VectorDBConfig connects to PGVECTOR_URL; checkpoint connects to CHECKPOINT_URL — two env vars, two RDS endpoints
 - Cost: two smaller instances vs one large instance is often cost-neutral; operational isolation is worth the added configuration
-

ALB, WAF & Bedrock Migration

Q07**Intermediate**

Explain the ALB and WAF v2 setup in your architecture. What does each layer do and what rules does your WAF ACL enforce?

Hint: ALB handles traffic distribution; WAF handles security — explain both.

Key points to cover:

- ALB (Application Load Balancer): terminates HTTPS (ACM certificate), performs path-based routing to ECS target groups, runs health checks every 30s
 - ALB access logs: sent to S3 — useful for debugging, rate analysis, and security auditing
 - WAF v2 ACL attached to ALB: inspects every HTTP request before it reaches ECS — runs before slowapi rate limiting
 - WAF rule 1: AWS managed rule groups — CommonRuleSet (SQLi, XSS, known bad IPs), AmazonIpReputationList (bot/scanner IPs)
 - WAF rule 2: Rate limiting rule — 1000 requests per 5 minutes per IP — coarse limit before application-level slowapi fine limits
 - WAF rule 3: Geo restriction — allow only specific countries if governance data is regionally restricted
 - WAF rule 4: Size constraint — block requests with body > 10MB (protect /ingest from accidental huge uploads before FastAPI sees them)
 - P0 item: WAF rate limiting as PRIMARY defence — disable slowapi memory:// fallback; WAF is the guaranteed rate limiter regardless of Redis state
-

Q08**Intermediate**

Your P0 migration is from Groq to AWS Bedrock (Claude 3.5 Sonnet). What specifically changes in the codebase, IAM, and network? Why is this a P0 priority?

Hint: P0 means highest priority — justify the urgency and explain the complete migration.

Key points to cover:

- P0 reason 1: data sovereignty — queries sent to Groq exit your AWS VPC to Groq's servers; Bedrock keeps all inference within AWS; critical for enterprise governance data
 - P0 reason 2: SLA alignment — Groq has no enterprise SLA guarantee; Bedrock has AWS SLA (99.9% uptime) matching your ECS/RDS commitments
 - P0 reason 3: IAM-based auth — no GROQ_API_KEY to rotate or protect; Bedrock uses ECS task role — credentials are ephemeral and automatically rotated
 - Code change: LLM_PROVIDER=bedrock, LLM_MODEL=anthropic.claude-3-5-sonnet-20241022-v2:0 in env vars; llm_factory.py creates a Bedrock client via LiteLLM — literally two env var changes
 - IAM change: ECS task role needs bedrock:InvokeModel permission for the specific model ARN — least-privilege scoping
 - Network change: Bedrock VPC endpoint (Interface endpoint) — Bedrock API calls route through the VPC endpoint, never touch the public internet
 - Bedrock VPC endpoint: create aws_vpc_endpoint for com.amazonaws.{region}.bedrock-runtime; attach to the VPC; update security groups to allow ECS → endpoint
 - Fallback chain update: Bedrock primary → keep gpt-4o-mini as first fallback (via internet egress or PrivateLink) → gemini last resort
-

Q09**Advanced**

Walk through the AWS Secrets Manager integration. What secrets does your system store there, how are they injected into ECS tasks, and what is the rotation strategy?

Hint: Secrets Manager is the right place for credentials — show you know how to wire it to ECS.

Key points to cover:

- Secrets stored: DATABASE_URL (Neon/RDS connection string), REDIS_AUTH token, OPENAI_API_KEY (embeddings), ANTHROPIC_API_KEY (Bedrock fallback), TEAMS_WEBHOOK_SECRET, COLLIBRA credentials, JIRA credentials
- ECS injection: ECS task definition references secrets via 'secrets' block — each secret maps to an env var name and a Secrets Manager ARN
- At task startup: ECS agent fetches the secret value from Secrets Manager and injects it as an env var inside the container — never stored on disk
- IAM permission: ECS task execution role needs secretsmanager:GetSecretValue for each secret ARN — least-privilege; task role can only read its own secrets
- Rotation strategy: DATABASE_URL — automatic rotation with RDS native rotation (Lambda rotates password, updates RDS and Secret simultaneously, zero downtime)
- Rotation strategy: API keys (OpenAI, JIRA) — manual rotation with versioning; Secrets Manager supports AWSPREVIOUS/AWSCURRENT/AWSPENDING stages during rotation
- During rotation: ECS tasks cache the secret in memory for the container lifetime; new tasks pick up the rotated value; old tasks gracefully drain (ALB connection draining)
- Audit: Secrets Manager logs every GetSecretValue call to CloudTrail — compliance teams can audit who accessed what credential and when

Observability, MWAA & Production Hardening

Q10**Advanced**

Your production observability is shifting from LangSmith to CloudWatch and X-Ray. What specific signals do you instrument and how does X-Ray trace a request across ECS tasks?

Hint: CloudWatch metrics + X-Ray distributed traces are the AWS-native observability stack.

Key points to cover:

- CloudWatch custom metrics (pushed via boto3.put_metric_data): agent_execution_ms per agent, cache_hit_rate per agent, intent_distribution (count per intent), token_budget_remaining, hitl_pending_count
- CloudWatch Logs: structured JSON logs from logging_utils.py → awslogs driver in ECS task definition → log group per service (api, app, mcp_server)
- CloudWatch alarms: p99 latency > 10s → PagerDuty alert; error rate > 5% → SNS notification; token_budget < 20% → Slack warning
- X-Ray: instrument with aws_xray_sdk; @xray_recorder.capture() on each agent node — creates a subsegment per agent with timing and metadata
- X-Ray trace propagation: ALB injects X-Amzn-Trace-Id header; FastAPI middleware reads and propagates it; LangGraph nodes continue the trace
- X-Ray service map: shows ALB → FastAPI → LangGraph → RDS/ElastiCache/Bedrock — visual latency breakdown per downstream service
- Replacing LangSmith: X-Ray captures node timing and errors; CloudWatch Logs capture the full AgentState at each node for debugging — equivalent observability within AWS
- Cost: CloudWatch + X-Ray is ~\$50-100/month at moderate volume vs LangSmith paid tier at ~\$200+/month; stays within AWS billing

Q11**Advanced**

An on-call engineer gets a PagerDuty alert at 2am: 'ECS api task count dropped from 2 to 0'. Walk through your incident response using the AWS toolchain.

Hint: This tests operational knowledge of AWS — not just architecture.

Key points to cover:

- Step 1: ECS Console or CLI — describe-services to see current task count, desired count, and recent deployment events; check if service is in DRAINING state
- Step 2: CloudWatch Logs — filter /ecs/api log group for ERROR in the last 15 minutes — identify the crash cause (OOM, unhandled exception, failed health check)
- Step 3: ECS Stopped Tasks — console shows stopped tasks with stop reason; 'Essential container in task exited' + exit code 137 = OOM Kill; exit code 1 = application crash
- Step 4: CloudWatch Metrics — check MemoryUtilization (OOM?), CPUUtilization (CPU spike?), ALB HealthyHostCount (confirms 0 healthy targets)
- Step 5: check Secrets Manager — if a secret was recently rotated, the old DATABASE_URL in running tasks may have expired; new tasks fail to start with auth errors
- Step 6: ECS force deployment — if a bad image was deployed, roll back: `aws ecs update-service --task-definition {previous-revision}`
- Step 7: RDS — if tasks are crashing on DB connection, check RDS CloudWatch for connection count (max_connections exceeded?) and slow query log
- Prevention: CloudWatch alarm on ECS HealthyHostCount < 1 → PagerDuty + auto-scaling trigger to replace failed tasks immediately

Q12**Design**

An interviewer asks: 'Your project started with Docker Compose. You moved to ECS Fargate. Your roadmap mentions EKS. What drives each migration and when is each the right choice?'

Hint: This is an infrastructure maturity question — show you understand the why behind each platform.

Key points to cover:

- Docker Compose (Day 1-19): single-host orchestration — zero cloud cost, fast iteration, perfect for development and demo; no HA, no scaling, no cloud-native integration
- ECS Fargate (current): managed container orchestration — AWS handles host provisioning, patching, and scaling; integrates natively with ALB, Secrets Manager, IAM, CloudWatch
- ECS is right when: team is small (1-3 engineers), workload is predictable, you want managed simplicity over control; Fargate removes EC2 management entirely
- ECS limitations: no pod-level networking policies (only security groups), limited scheduling flexibility, no native service mesh, Helm charts don't apply
- EKS (roadmap): Kubernetes — full control over scheduling, networking (Calico, Istio), autoscaling (HPA, KEDA), and ecosystem (Helm, ArgoCD, Prometheus)
- EKS is right when: multiple teams deploy independently, complex microservice mesh, need GPU nodes for self-hosted embedding models, or Kubernetes expertise already exists
- Migration trigger: EKS makes sense when ECS scaling or networking limitations are actually felt — premature migration adds operational burden without benefit
- Honest answer: for this project's scale (3 services, 1 team), ECS Fargate is the correct long-term platform; EKS is aspirational and would require dedicated DevOps investment